SETUP GUIDE · OCTOBER 2026

Run both systems on your own PC

From a clean Windows 11 PC to a working setup: tools, two Python environments, local models (Whisper, Qwen, Demucs, ECAPA), path settings, first runs for a YouTube film, an Instagram Reel, long-video-to-shorts and the Telegram bot, then the Obsidian context harness shared across PCs.

1–2 h	**≈25 GB**	**16 GB / 6 GB**
mostly downloads	tools and models	RAM / NVIDIA VRAM (CPU-only works, slower)

Same content as SETUP.md in the repository.

This guide takes a clean Windows 11 PC to a working setup: analysing a YouTube video, an Instagram Reel or a TikTok frame by frame, turning a long stream into short clips, receiving links through your own Telegram bot, and keeping an AI assistant's memory in an Obsidian vault shared across PCs.

Every step names the exact command. Where the code still holds the authors' machine paths, section 6 lists each one and what to change it to.

Time: about 1–2 hours, mostly downloads. **Disk:** about 25 GB (tools + models) plus space for videos.

0. What you need

Item	Minimum	What the authors use
OS	Windows 10/11 64-bit (Linux and macOS work with the notes in each step)	Windows 11
RAM	16 GB	16 GB
GPU	NVIDIA with 6 GB VRAM (CPU-only works, but is much slower)	RTX laptop GPU, 6–8 GB
Disk	50 GB free	
Accounts	GitHub (free); Claude Pro or Max for the Claude steps; Telegram for the bot (optional); Instagram only for Reels	

Models you will download (once):

Model	Size	Used for
faster-whisper `large-v3` (or `large-v3-turbo` / `small` for speed)	3.1 GB (1.6 GB / 0.5 GB)	Speech to text with word times
Qwen3-VL 8B via Ollama	about 6 GB	Describing frames
Qwen3 4B via Ollama	about 2.5 GB	Backup judge, voice check
Demucs `htdemucs`	about 80 MB	Voice / music split
SpeechBrain ECAPA	about 80 MB	Speaker fingerprints

1. Get the code

```
winget install --id Git.Git -e --source winget
cd <a folder for your code>
git clone https://github.com/yushaaanan-design/ai-systems-portfolio.git ai-systems-portfolio
cd ai-systems-portfolio
```

No Git? Click **Code** → **Download ZIP** on GitHub and unzip it to a folder of your choice (called `<repo folder>` below).

> Keep code outside OneDrive and outside any Obsidian vault. Syncing services lock files mid-run and upload multi-GB videos.

2. Install the base tools

Open **PowerShell** (no admin needed for most of these) and run one line at a time:

```
winget install --id Python.Python.3.13 -e --source winget      # Python + the "py" launcher
winget install --id Gyan.FFmpeg -e --source winget              # FFmpeg + ffprobe
winget install --id OpenJS.NodeJS.LTS -e --source winget        # Node.js: yt-dlp needs it for
YouTube
winget install --id Ollama.Ollama -e --source winget            # runs Qwen locally
winget install --id Obsidian.Obsidian -e --source winget        # the note vault (harness)
winget install --id GitHub.cli -e --source winget               # optional: gh, for private repos
```

Then **close and reopen PowerShell** so the new commands are on your PATH.

NVIDIA driver: install the latest Game Ready or Studio driver from nvidia.com. You do **not** need the CUDA toolkit: the GPU libraries come from pip in step 3.

Claude Code (for the "Claude reads the summary" steps and the clipper's judge/writer):

```
npm install -g @anthropic-ai/claude-code
claude          # first run opens a browser to log in with your Claude account
```

The clipper calls `claude -p` in the background, so you must have logged in once on this PC.

Linux (Ubuntu)	macOS (Homebrew)
`sudo apt install git python3.13 python3.13-venv ffmpeg nodejs npm`	`brew install git python@3.13 ffmpeg node ollama`
Ollama: `curl -fsSL https://ollama.com/install.sh | sh`	Obsidian: `brew install --cask obsidian`

Check:

```
py -3.13 --version        # Python 3.13.x
ffmpeg -version           # first line: ffmpeg version ...
node -v                   # v20 or newer
ollama --version
nvidia-smi                # shows your GPU and free memory
```

3. Create the two Python environments

Two environments, because Demucs and SpeechBrain pin their own PyTorch build, which can clash with the main tools.

```
cd <repo folder>

# main environment
py -3.13 -m venv .venv
.\.venv\Scripts\python -m pip install --upgrade pip
.\.venv\Scripts\python -m pip install -r requirements.txt

# audio environment
py -3.13 -m venv .venv-audio
.\.venv-audio\Scripts\python -m pip install --upgrade pip
.\.venv-audio\Scripts\python -m pip install torch==2.8.0 torchaudio==2.8.0 --index-url
https://download.pytorch.org/whl/cu128
.\.venv-audio\Scripts\python -m pip install -r requirements-audio.txt
```

- No NVIDIA GPU? Use `--index-url https://download.pytorch.org/whl/cpu` for torch and skip the two `nvidia-*` lines in `requirements.txt`.
- "running scripts is disabled on this system": run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, or always call `.\.venv\Scripts\python` directly as shown (no activation needed).
- The film scripts call `py -3.13`. To use the venv instead, replace `py -3.13` with `<repo folder>\.venv\Scripts\python` in `run_film.ps1`, or install the requirements into the `py -3.13` Python as well.

Check:

```
.\.venv\Scripts\python -c "import yt_dlp, faster_whisper, scenedetect, cv2, truststore, psutil,
feedparser; print('main OK')"
.\.venv-audio\Scripts\python -c "import demucs, speechbrain, soundfile, scipy, torch; print('audio
OK, CUDA:', torch.cuda.is_available())"
```

4. Download the models

Whisper (the film script loads it with `local_files_only=True`, so download it once first):

```
.\.venv\Scripts\python -c "from faster_whisper import WhisperModel; WhisperModel('large-v3',
device='cpu', compute_type='int8'); print('whisper ready')"
```

Qwen models through Ollama (Ollama runs in the background after install; the tray icon shows it):

```
ollama pull qwen3-vl:8b      # vision: describes frames
ollama pull qwen3:4b         # text: backup judge and voice check
ollama list
```

The authors used community builds of the same models (`huihui_ai/qwen3-vl-abliterated:8b-instruct`, `huihui_ai/qwen3-abliterated:4b`). The official ones above work the same way. Set the names here:

File	Line	Set to
`video-pipeline/2-film-analysis/tag.py`	`MODEL = ...`	`"qwen3-vl:8b"`
`video-pipeline/2-film-analysis/run_film.ps1`	`ollama stop ...`	`ollama stop qwen3-vl:8b`
`video-pipeline/3-shorts-clipper/moments/finder.py`	`JUDGE_MODEL = ...`	`"qwen3:4b"`

Speaker model (SpeechBrain ECAPA, into the folder `voices.py` expects):

```
.\.venv-audio\Scripts\python -c "from huggingface_hub import snapshot_download;
snapshot_download('speechbrain/spkrec-ecapa-voxceleb', local_dir=r'video-pipeline\2-film-
analysis\models\ecapa'); print('ecapa ready')"
```

Demucs downloads `htdemucs` by itself on its first run.

Quick model test:

```
ollama run qwen3:4b "Reply with the single word: ready"
```

5. Folder layout after setup

```
<repo folder>\
  .venv\  .venv-audio\
  video-pipeline\
    2-film-analysis\
      models\ecapa\          speaker model (step 4)
      video\                 downloaded films land here (dl.py)
      work\<video_id>\       every output for one film
      cookies.txt            OPTIONAL, your exported cookies (never commit)
    3-shorts-clipper\
      work\<video_id>\       long video, transcript, clips\
      data\assistant.db      job queue (created on first run)
```

`video\`, `work\`, `models\`, databases, cookies and tokens are already in `.gitignore`.

6. Point the scripts at your PC

The scripts are shown exactly as they ran, so a few constants still name the authors' folders. Change these once (open the file in any editor, search for the line).

File	Constant / line	Authors' value	Change to
`2-film-analysis/run_film.ps1`	`$root`	the authors' project folder	`<repo folder>\video-pipeline\2-film-analysis`
`2-film-analysis/run_film.ps1`	`$vs`	another app's Python	`<repo folder>\.venv-audio\Scripts\python.exe`
`2-film-analysis/run_film.ps1`	`$ff`	winget FFmpeg folder	keep it if you installed FFmpeg with winget; otherwise the folder holding `ffmpeg.exe`
`2-film-analysis/run_film.ps1`	`-c:v h264_nvenc -preset p4 -cq 20`	NVIDIA encoder	no NVIDIA GPU: `-c:v libx264 -preset veryfast -crf 20`
`2-film-analysis/dl.py`	the FFmpeg `glob`	winget folder	same rule as `$ff`
`1-intake/reel_bot.py`	`OUT`, `COOKIES`, `YTDLP`, `FFMPEG`	the authors' folders, a separate venv	your inbox folder, your cookie file, `.venv\Scripts\python.exe`, your FFmpeg `bin` folder
`1-intake/loop.py`, `report.py`, `reellog.py`, `screentext.py`, `collect_links.py`	`ROOT`, `ANALYZE`, `OUT`, `FFMPEG`, `VAULT_DIGESTS`	the authors' folders and vault	your folders (see the note in 7.5)
`3-shorts-clipper/config/sources.json`	`brand_voice_note`, `playbook_note`, `review_dir`, `digest_dir`	vault paths	notes in your own vault (missing notes are simply skipped)
`3-shorts-clipper/config/sources.json`	`facecam`	one streamer's webcam box	the x/y/w/h of the webcam in your source video, or delete it for the blurred layout
`3-shorts-clipper/chief/core.py`	`notify()` bot path	the bot folder	your `1-intake` folder, or set `"notify_telegram": false` in `config/budget.json`
`3-shorts-clipper/writer/writer.py`	`CLIPS_DB`	the authors' study database	leave it: a missing database is handled

7. Run it

7.1 Analyse one YouTube video, frame by frame

```
cd <repo folder>\video-pipeline\2-film-analysis
.\run_film.ps1 -Id <youtube_id> -Stage cpu     # download, audio, proxy, cuts + motion, contact sheets
.\run_film.ps1 -Id <youtube_id> -Stage gpu     # Whisper, Demucs, audio report, voices, Qwen frame tags
py -3.13 .\film_card.py .\work\<youtube_id>     # the one-page summary
```

`<youtube_id>` is the part after `v=` in the link (`https://www.youtube.com/watch?v=jNQXAC9IVRw` → `jNQXAC9IVRw`).

Two stages so the GPU holds one heavy model at a time. Run the GPU stage when nothing else uses the GPU.

What you get in `work\<id>\`:

File	What it holds
`film_card.md`	Every number on one page, by chapter: cuts per minute, words per minute, music level, camera moves, media mix, voices, longest pauses and the line before each
`transcript.txt` / `transcript.json`	Every sentence with its time / every word with start and end
`shots.csv`	One row per shot: start, length, brightness, camera move, 5-colour palette
`sheets\sheet_NN.jpg`	48 shots per image, for reading the film by eye
`audio_report.json`, `voices.json`, `tags.jsonl`	The raw measurements behind the card

Need a closer look at a few shots? `py -3.13 .\sheets.py .\work\<id> strip 12 40 133` writes 8 frames across each of those shots.

7.2 Analyse an Instagram Reel or a TikTok

`run_film.ps1` only downloads from YouTube by itself. For other sites, download first with `dl.py` (it names the file by the post's id), then run the same stages:

```
py -3.13 .\dl.py "https://www.instagram.com/reel/<code>/" .\video
.\run_film.ps1 -Id <code> -Stage cpu
.\run_film.ps1 -Id <code> -Stage gpu
py -3.13 .\film_card.py .\work\<code>
```

Instagram needs your logged-in session as a cookie file (section 8). Save it as `2-film-analysis\cookies.txt`, which is where `dl.py` looks. Public YouTube videos work without it.

7.3 Let Claude write the study document

Open Claude Code in the work folder and paste a prompt like this:

```
cd <repo folder>\video-pipeline\2-film-analysis\work\<id>
claude
```

```
Read film_card.md and transcript.txt, then look at sheets/sheet_00.jpg to the last sheet.
Do not open frames one by one or read the json/npz files.
Write study-notes.md: a 10-line summary, the structure with timestamps, the key ideas,
how it is edited (pace, cuts, camera, sound) with numbers from the card, and 5 questions to test
understanding.
Label anything you infer as a guess.
```

This is the step that keeps the cost at about 18,000 tokens for an 18-minute film instead of hundreds of thousands.

7.4 Long video → short clips

```
cd <repo folder>\video-pipeline\3-shorts-clipper
..\..\.venv\Scripts\python -X utf8 chief\run_all.py --selftest                  # checks the queue
and lock
..\..\.venv\Scripts\python -X utf8 ingest\prepare_vod.py "<youtube link>"       # download +
transcript + loudness, queues the jobs
..\..\.venv\Scripts\python -X utf8 chief\run_all.py                             # moments -> writer
-> cutter
```

`prepare_vod.py` options: `--model large-v3-turbo` (faster), `--cookies <file>`, `--top 3` (how many clips to cut).

What you get in `work\<video_id>\`:

- `sheets\top10.jpg`: three frames for each of the 10 best moments, with scores
- `clips\01_<start>.mp4` ...: 1080×1920 draft clips with word-highlighted captions and a pinned label
- The scores, reasons, hooks, titles and captions, in `data\assistant.db` (tables `candidates`, `drafts`, `clips`)

Nothing is posted anywhere. Watch the clips and decide yourself. The windows are 25–62 s, so use a source video of at least 10 minutes.

If Claude is not logged in or the nightly budget (`config\budget.json`) is spent, the judge falls back to the local Qwen model, and the writer step reports "no copy written". The clips are still cut.

7.5 Your own Telegram bot (send links from your phone)

Step 1. In Telegram, open **@BotFather**, send `/newbot`, choose a name and a username ending in `bot`.

Step 2. BotFather replies with a token. Save it **only** in `video-pipeline\1-intake\.telegram-token` (one line, no quotes). Never paste it in a chat or commit it.

Step 3. Set `OUT`, `COOKIES`, `YTDLP` and `FFMPEG` at the top of `reel_bot.py` (section 6).

Step 4. Test the link reader, then start the bot:

```
cd <repo folder>\video-pipeline\1-intake
..\..\.venv\Scripts\python reel_bot.py --selftest     # prints "selftest ok"
..\..\.venv\Scripts\python reel_bot.py
```

Step 5. Open `pair-code.txt` in the same folder and send `/start` followed by that code to your bot. From then on it obeys only you.

Step 6. Send any Instagram, YouTube or TikTok link (or a video under 20 MB). The file lands in `OUT`. `/status` shows the cookie age and whether Instagram is paused.

Step 7. Analyse it with 7.1 / 7.2.

`loop.py` (collect saved Instagram collections → download → analyse, every N minutes) is included as reference. Its analysis step calls the authors' earlier single-reel analyser, `analyze_reel.py`, which is not in this repository. Use 7.2 for analysis instead. `collect_links.py` needs `python -m playwright install chromium` once.

7.6 The clip study (reference)

`4-clip-study/cycle.py` is the resumable study that measured 1,000+ Shorts. Like `loop.py`, it calls `analyze_reel.py`, so treat it as reference code showing the method: queue, polite pacing, STOP file, delete video after analysis, comments without usernames.

8. Cookies, safely (only for Instagram or age-gated videos)

1. In Chrome, install the open-source extension **Get cookies.txt LOCALLY** (from the link in its own GitHub README).
2. Log in to the site normally in Chrome. Then, on that site, click the extension and **Export** the cookies for that site only.
3. Save the file where the script looks (`2-film-analysis\cookies.txt` , or the `COOKIES` path in `reel_bot.py`).
4. Treat it like a password: it lets anyone act as your account. Never commit it (it is git-ignored), never paste it into a chat. To cut access, log out of the site in Chrome and delete the file.

Pacing the code already enforces: one download at a time, 30 s between Instagram downloads, a 10-minute pause on HTTP 429, and a full stop with a `STOP` file on a login challenge, "sign in to confirm" or 403. If a `STOP` file appears, read it, wait, and delete it by hand before running again. Do not try to get around a block.

9. Set up the context harness (the assistant's memory)

9.1 Make the vault

Step 1. Open Obsidian → **Create new vault** → name it, place it **outside** your code folder (e.g. `<your vault>`).

Step 2. Create this structure and copy the templates from `context-harness\templates\` :

```
<your vault>\
  INDEX.md                       a list of projects, one line each
  Harness\Preferences.md         how you want the assistant to work
  Harness\Lessons.md             symptom | cause | fix
  Harness\Eval Checklist.md      what must be true before "done"
  Harness\Vault Registry.md      every vault and where to resume it
  Projects\<project>\STATE.md    from templates\STATE.md
  Projects\<project>\run-log.md  from templates\run-log.md
  _Harness\Resume Protocol.md    from templates\Resume Protocol.md
```

Step 3. Start every note with front matter naming its parent, e.g. `up: "[[INDEX]]"` , and link related notes with `[[Note name]]` . Open **Graph view**: that graph is the ontology.

Step 4. Optional: **Settings** → **Graph** → **Groups**, one colour per project (query `path:"Projects/<name>"`).

9.2 Tell Claude Code to use it

Create `%USERPROFILE%\.claude\CLAUDE.md` (Claude Code loads it in every session) with the rules from `context-harness\CLAUDE.example.md` . Put in your vault's real paths. The key rules: never rebuild context from the chat, read Preferences → Registry → Resume Protocol → STATE → the end of run-log, write back to run-log/STATE after each step, add a Lessons row for every mistake, and run the Eval Checklist before saying done.

Test it: open a new Claude Code session and ask "what is the state of `my-project` ?". It should read the notes and cite them.

9.3 Share it across PCs and accounts

```
cd "<your vault>"
git init -b main
Add-Content .gitignore ".obsidian/workspace*.json`n.trash/"
git add -A; git commit -m "First vault commit"
gh repo create my-vault --private --source . --push        # or create a private repo on github.com
and git push
```

On every other PC: install Obsidian, Git and Claude Code, `git clone` the private repo, open it as a vault, and copy the same `CLAUDE.md`. Any Claude account signed in on any of those PCs now starts from the same notes.

Daily sync (pull, then commit and push only changed notes after a secret scan; stops on a conflict or a secret):

```
# Git Bash
bash <repo folder>/context-harness/scripts/sync-vaults.sh MyLaptop "<your vault>"
```

Keep the global rules file in a private repo too, so every machine loads the same rules.

9.4 Optional: a generated ontology vault

`context-harness\ontology-generator\ontology.py` is a map of domains → concepts (keyword pattern + one-line definition). `build_vault.py` matches collected audience questions (JSON lines in `data\youtube`, `data\reddit`, `data\x`, `data\exa`) to those concepts and writes one linked note per concept plus an `Ontology.canvas`. It is included to show the method. Its collectors are not part of this repository, so feed it your own question files in the same shape or adapt `load()`.

10. Troubleshooting

Symptom	Cause	Fix
`CERTIFICATE_VERIFY_FAILED` on downloads	Antivirus re-signs HTTPS (some antivirus products)	Already handled with `truststore` in `dl.py` / `ytdl.py`. For other scripts add `import truststore; truststore.inject_into_ssl()` at the top. Never switch certificate checks off
`Library cublas64_12.dll is not found`	Windows does not see pip's GPU libraries	`pip install nvidia-cublas-cu12 nvidia-cudnn-cu12`; the scripts add those folders at start-up (top of `transcribe.py`)
`open() got an unexpected keyword argument 'metadata_errors'`	PyAV 19 vs faster-whisper's own decoder	Pass audio as a numpy array (as `transcribe.py` and `prepare_vod.py` do), or `pip install "av<19"`
YouTube: "Sign in to confirm you're not a bot"	Too many requests, or a datacentre IP	Stop, wait some hours, run from a home connection; add your cookies (section 8). Do not loop retries
YouTube: no formats / "n challenge" errors	yt-dlp outdated or Node.js missing	`.\.venv\Scripts\python -m pip install -U "yt-dlp[default]"`; check `node -v`
Instagram: "login required" / checkpoint	Session expired or a security check	Open Instagram in Chrome, resolve the check, export cookies again, restart the bot
`yt-dlp.exe` blocked by Windows	Application Control / Smart App Control	Use `python -m yt_dlp` (the scripts already do)
Demucs or Whisper: CUDA out of memory	Another model holds the GPU	`ollama stop <model>`; close games and browsers; run the GPU stage alone
A job killed / "system low on memory"	16 GB RAM shared with Chrome	One heavy job at a time; close Chrome; the clipper waits for free RAM by itself (`config\budget.json`)
Nothing downloads with `--print`	`--print` implies "simulate"	Add `--no-simulate` (the bot already does)
`run_film.ps1` cannot be loaded	PowerShell execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
Qwen tags look wrong for a whole category	The prompt's labels do not fit your videos	Check 5+ frames per label by hand first; add a one-line definition per label in `tag.py`'s prompt
`ModuleNotFoundError: feedparser`	Main requirements not installed	`.\.venv\Scripts\python -m pip install -r requirements.txt`

11. Updating and removing

- Update tools: `.\.venv\Scripts\python -m pip install -U "yt-dlp[default]"` (do this first whenever downloads break); `winget upgrade --all`; `ollama pull <model>` again for a newer build.
- Free disk: delete `work\<id>\proxy.mp4`, `audio*.wav` and `stems\` after a film card is written; delete videos in `video\`.
- Remove models: `ollama rm qwen3-vl:8b`; delete `%USERPROFILE%\.cache\huggingface\hub\models--Systran--faster-whisper-*`.

- Remove everything: delete `<repo folder>` and uninstall the winget packages.

12. Rules worth keeping

- Local models do the bulk; Claude reads only the compact outputs. State the cost before a big run.
- Check the real output (open the clip, look at the sheet) before calling anything done.
- Test any model used as a judge on a clearly good and a clearly bad example before trusting it.
- Secrets (bot token, cookies) live in files that are git-ignored, are never printed, and are never pasted into a chat.
- Respect the platforms: slow, one at a time, stop on a block, keep only what you need.